
Introduction à l'algorithme k-means avec Mahout

Code de l'UE:	NFE204
Version:	01.00 / FR
Auteur:	Boris GUARISMA
Date:	28/02/2014

VERSIONS

Version	Date	Auteur	Contact
01.00	28/02/2014	Boris Guarisma	boris.guarisma@outlook.com

SOMMAIRE

1.	INTRODUCTION.....	5
1.1.	OBJECTIF	5
1.2.	LIMITES DE L'ETUDE.....	5
1.3.	AUDIENCE	6
2.	INTRODUCTION À APACHE MAHOUT	7
2.1.	L'APPRENTISSAGE AUTOMATIQUE	7
2.2.	LE PROJET MAHOUT	8
2.2.1	Algorithmes de Classification	8
2.2.2	Algorithmes de Filtrage Colaboratif.....	9
2.2.3	Algorithmes de Clustering	9
3.	CLUSTERING	11
3.1.	CONCEPTS	11
3.1.1	Ambiguïté des termes	11
3.1.2	Partitionnement	11
3.1.3	Classification Non Supervisée	12
3.1.4	Classification Non Hiérarchique	12
3.2.	NOTION DE SIMILARITÉ OU DE DISTANCE.....	13
3.2.1	Mesures de Distance	13
3.3.	EXEMPLES D'APPLICATIONS	14
4.	L'ALGORITHME K-MEANS	15
4.1.	INTRODUCTION	15
4.1.1	Pseudo-Code	15
4.2.	LIMITATIONS	15
4.2.1	Choix des représentants initiaux	15
4.2.2	Condition d'arrêt et convergence	16
4.2.3	Complexité	17
4.3.	SOLUTIONS	17
4.3.1	Canopy Clustering.....	17
4.3.2	Bisecting k-means.....	18
4.3.3	MapReduce	19
5.	K-MEANS AVEC APACHE MAHOUT	19
5.1.	LA COMMANDE KMEANS	19
5.2.	PROCÉDURE DE BOUT EN BOUT	20
5.2.1	Préparation.....	21
5.2.2	Vectorisation.....	22
5.2.3	Exécution de K-means	25
5.2.4	Inspection	26
6.	K-MEANS ET MAPREDUCE.....	29
6.1.	DESIGN MAHOUT	29

6.2.	AUTRES DESIGNS	30
6.2.1	Résultats Expérimentaux	31
7.	CONCLUSION	32
8.	ANNEX A – CONFIGURATION.....	33
9.	BIBLIOGRAPHIE ET SOURCES	33
9.1.	OUVRAGES.....	33
9.2.	LIVRES BLANCS TECHNIQUES.....	34
9.3.	SOURCES WEB.....	34

TABLE DE FIGURES

Figure 1 - Système de recommandations (Amazon).....	9
Figure 2 - Regroupement d'articles de presse (Yahoo!)	10
Figure 3 - Mesure de distance Manhattan.....	13
Figure 4 - Exemple d'un mauvais partitionnement ou clustering	16
Figure 5 - Algorithme Canopy Clustering	18
Figure 6 - Procédure de bout en bout	20
Figure 7 - Fichiers SGML (articles Reuters).....	22
Figure 8 - seqdirectory: fichier résultat en format SequenceFile.....	23
Figure 9 - Structure du répertoire généré par seq2sparse.....	25
Figure 10 - Structure du répertoire généré par kmeans	26
Figure 11 - Top termes avec distances euclidienne et cosinus	28
Figure 12 - Une itération Map et Reduce de k-means	29
Figure 13 - Résultats expérimentaux [Zhao et al. 2009]	31

1. INTRODUCTION

1.1. OBJECTIF

Le présent document est un **rapport de projet** correspondant à l'unité d'enseignement NFE204 « Base de données avancées 1 » dispensée au Conservatoire Nationale des Arts et Métiers (CNAM) de Paris.

Ce rapport peut être présenté comme une **prise en main** du *framework* Apache Mahout à travers l'implémentation de l'algorithme de *clustering* k-means.

Le chapitre 2 fournit une introduction rapide à l'apprentissage automatique (supervisé et non supervisé) et présente le projet Apache Mahout ainsi que les algorithmes (de classification, de filtrage collaboratif et de clustering) supportés par Mahout au moment de l'écriture de ce rapport.

Le chapitre 3 explique le clustering en tant qu'ensemble de méthodes d'apprentissage automatiques non supervisés. Après quelques clarifications par rapport aux termes utilisés dans le domaine du partitionnement, le chapitre explique des notions clés telles que la similarité et les mesures de distances supportées par Mahout afin de comprendre les principes d'une méthode de *clustering*.

Le chapitre 4 présente l'algorithme k-means, propose un pseudo code et explique les limitations de cet algorithme. L'une de ces limitations étant sa complexité et elle sera résolue avec le paradigme MapReduce sur lequel se base la plupart des algorithmes de Mahout. D'autres limitations et solutions respectives, telles que les algorithmes *canopy clustering* et le *bisecting k-means*, seront présentées.

Une fois le contexte expliqué par les chapitres 3 et 4, le chapitre 5 présente enfin l'implémentation de l'algorithme k-means avec Mahout sans la notion de MapReduce (qui sera présentée au chapitre 6). Un exemple de partitionnement de documents via la vectorisation de documents texte (articles de l'agence de presse Reuters) permettra de faire un rappel des notions « recherche d'information » apprises durant le cours NFE204 telles que la pondération TF-IDF.

Finalement le chapitre 6 présente la parallélisation des traitements de l'algorithme k-means avec le modèle de programmation MapReduce (MR). Il explique sommairement le design MR de Mahout via la définition des classes Java existantes. Un autre exemple de design MR ainsi que ses résultats en termes de performances seront présentés.

1.2. LIMITES DE L'ETUDE

En plus d'une prise en main du *framework* Apache Mahout ce document est considéré également comme le résultat d'une recherche bibliographique sur le clustering et l'algorithme k-means plus spécifiquement.

Ce document ne couvre pas la démarche pour une amélioration de la qualité du partitionnement, il ne présente pas une comparaison entre les résultats de différentes mesures de similarité ou de distance par exemple.

Les aspects performances, optimisation et distribution sur Hadoop n'ont pas été développés d'un point de vue pratique même si l'exécution de k-means a été réalisée avec une configuration pseudo-distribuée de Hadoop. Le lecteur trouvera des informations sur l'environnement logiciel et matériel applicable au chapitre 5 dans l'annexe A.

Les aspects qualité de partitionnement et optimisation seront développés dans le cadre d'un deuxième projet à l'initiative de l'auteur.

1.3. AUDIENCE

Ce rapport est destiné à toute personne intéressée par l'apprentissage automatique et plus spécifiquement à l'implémentation des algorithmes avec le *framework* Apache Mahout. Pour tout public externe ou non au CNAM.

2. INTRODUCTION A APACHE MAHOUT

2.1. L'APPRENTISSAGE AUTOMATIQUE

Reprenons la définition à partir de la page Wikipédia : « l'apprentissage automatique (*machine learning* en anglais), un des champs d'étude de l'intelligence artificielle, est la discipline scientifique concernée par le développement, l'analyse et l'implémentation de méthodes automatisables qui permettent à une machine (au sens large) d'évoluer grâce à un processus d'apprentissage, et ainsi de remplir des tâches qu'il est difficile ou impossible de remplir par des moyens algorithmiques plus classiques.

Les algorithmes d'apprentissage peuvent se catégoriser selon le mode d'apprentissage qu'ils emploient :

L'apprentissage supervisé: si les classes sont prédéterminées et les exemples connus, le système apprend à classer selon un modèle de classement. Le processus se passe en deux phases. Lors de la première phase (hors ligne, dite d'apprentissage), il s'agit de déterminer un modèle des données étiquetées. La seconde phase (en ligne, dite de test) consiste à prédire l'étiquette d'une nouvelle donnée, connaissant le modèle préalablement appris. Parfois il est préférable d'associer une donnée non pas à une classe unique, mais une probabilité d'appartenance à chacune des classes prédéterminées.

L'apprentissage non-supervisé (ou classification automatique). Le système ou l'opérateur ne dispose que d'exemples, mais non d'étiquettes, et le nombre de classes et leur nature n'ont pas été prédéterminés. Aucun expert n'est requis. L'algorithme doit découvrir par lui-même la structure plus ou moins cachée des données. Le partitionnement de données est un algorithme d'apprentissage non supervisé ».

L'algorithme k-means présenté dans ce document est un algorithme d'apprentissage non supervisé.

La classification automatique non supervisée sera abordée en détail dans le chapitre 3.

L'ouvrage [Harrington 2012] propose un tableau (voir ci-dessous) classifiant les algorithmes les plus connus du domaine de l'apprentissage automatique. On présente les noms de ces algorithmes en anglais afin de les comparer avec ceux supportés par Apache Mahout et listés au chapitre 2.2.

Apprentissage Supervisé	
Classification	Régression
<i>k-Nearest Neighbors (KNN)</i>	<i>Linear</i>
<i>Naive Bayes</i>	<i>Locally Weighted Linear</i>
<i>Support Vector Machines (SVM)</i>	<i>Ridge</i>
<i>Decision trees</i>	<i>Lasso</i>

Apprentissage Non supervisé	
Clustering	Estimation de fonction de densité
<i>k-means</i>	<i>Expectation Maximization</i>
<i>DBSCAN</i>	<i>Parzen Window</i>

2.2. LE PROJET MAHOUT

Apache Mahout est un projet de la **Fondation Apache** visant à créer des implémentations d'algorithmes d'apprentissage automatique distribués sur leur plateforme Hadoop.

Mahout est une bibliothèque *open source* écrite en Java, c'est un framework d'outils pour l'implémentation d'algorithmes d'apprentissage automatique.

Même si les algorithmes principaux proposés par Mahout sont implémentés avec Apache Hadoop en utilisant le paradigme MapReduce, les contributions ne sont pas restreintes à une base Hadoop, mais peuvent aussi être non distribuées ou utiliser un cluster ne tournant pas sur Hadoop. Cela permet l'intégration de projets externes dans Mahout.

La dernière version d'Apache Mahout au moment de l'écriture de ce rapport est la **version 0.9**.

Le nombre d'algorithmes implémentés a rapidement augmenté depuis son début en 2009, mais certains manquent encore. On peut distinguer 3 grandes familles d'algorithmes proposés par Mahout: de **classification**, de **filtrage collaboratif** et de **clustering**.

2.2.1 ALGORITHMES DE CLASSIFICATION

On se place dans le domaine de l'apprentissage supervisé qui peut être présentée comme « une technique d'apprentissage automatique où l'on cherche à produire automatiquement des règles à partir d'une base de données d'apprentissage contenant des *exemples* (en général des cas déjà traités et validés) ».

L'exemple le plus populaire dans la bibliographie de Mahout, est le **détecteur des spams** des services de messageries (mails) tels que Gmail, Outlook, Yahoo Mail, etc.

Les algorithmes de « classification » supportés par Mahout sont :

- Logistic Regression
- Naive Bayes/ Complementary Naive Bayes
- Random Forests
- Hidden Markov Models

2.2.2 ALGORITHMES DE FILTRAGE COLLABORATIF

Reprenons la définition à partir de la page Wikipédia : « Le filtrage collaboratif regroupe l'ensemble des méthodes qui visent à construire des systèmes de recommandation utilisant les opinions et évaluations d'un groupe pour aider l'individu.

Il existe deux principaux axes de recherche dans le domaine :

Le **filtrage collaboratif actif** : qui repose sur du déclaratif (notes, commentaires) explicite de la part des utilisateurs. Cet axe de recherche se subdivise en deux autres catégories :

- le filtrage collaboratif utilisateurs
- le filtrage collaboratif objets

Le **filtrage collaboratif passif** : qui repose sur une analyse des comportements utilisateurs faite en "arrière-plan" de manière implicite.

Le **filtrage basé sur le contenu** qui consiste à définir des caractéristiques "objectives" aux objets afin de procéder au filtrage ».

Le système de filtrage collaboratif a été popularisé par Amazon avec la fonctionnalité « *les gens qui ont acheté x ont aussi acheté y* » (figure 1).

Les clients ayant acheté cet article ont également acheté



Figure 1 - Système de recommandations (Amazon)

Les algorithmes de filtrage collaboratif supportés par Mahout sont :

- *Non-distributed recommenders ("Taste")*
- *Distributed Item-Based Collaborative Filtering*
- *Collaborative Filtering using a parallel matrix factorization*

2.2.3 ALGORITHMES DE CLUSTERING

La classification automatique ou non supervisée sera abordée en détail dans le chapitre 3.

Les algorithmes de *clustering* supportés par Mahout sont :

- *Canopy Clustering*
- ***K-Means Clustering***
- *Fuzzy K-Means*

- *Mean Shift Clustering*
- *Dirichlet Process Clustering*
- *Latent Dirichlet Allocation*
- *Minhash Clustering*
- *kMeans++ streaming clustering*

Un exemple peut être le regroupement d'articles web provenant de différentes sources selon un sujet d'actualité (figure 2)

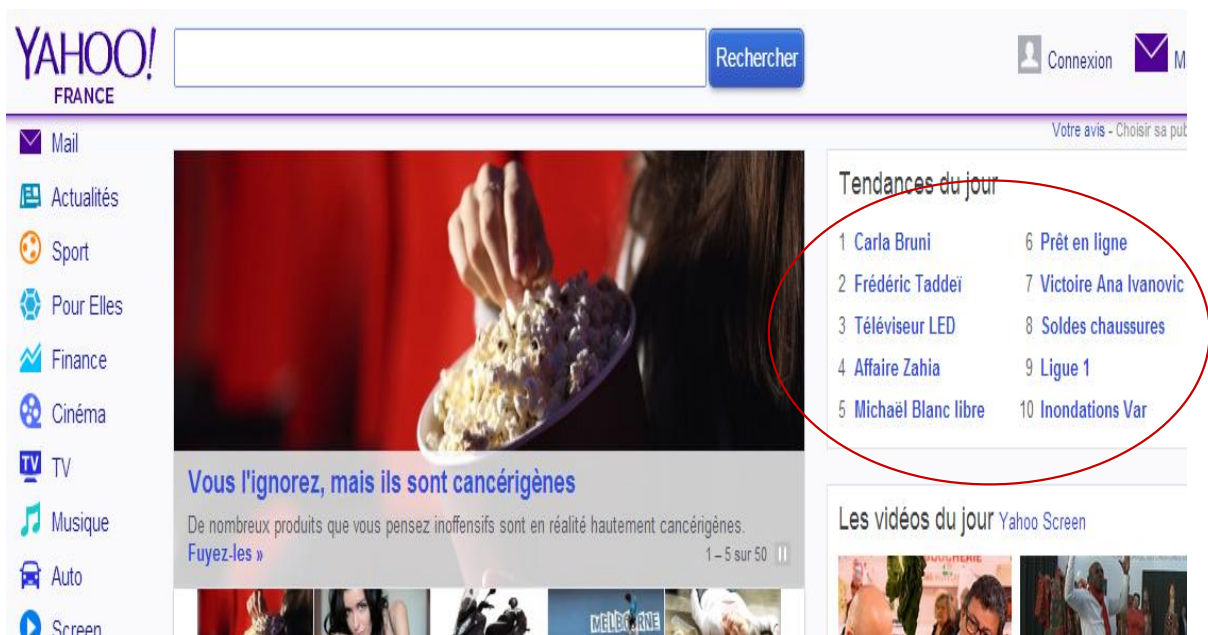


Figure 2 - Regroupement d'articles de presse (Yahoo!)

3. CLUSTERING

Le clustering possède un large spectre et les méthodes de partitionnement peuvent être regroupées selon leur implémentation de la manière suivante [Anchalia et al. 2013] :

- Techniques basées sur la connectivité, exemple : *Hierarchical Clustering*
- Techniques basées sur des centroïdes, exemple : ***K-Means Clustering***
- Techniques basées sur la distribution, exemple : *Expectation Maximization*
- Techniques basées sur la densité, exemple : *DBSCAN*
- Techniques basées sur des sous-espaces, exemple : *Co-clustering ou BI-Clustering* (ou classification double)

AVANTAGES

Le livre blanc [Anchalia et al. 2013] liste également quelques avantages du clustering :

- Il fournit une vue d'ensemble rapide et significative des données.
- Il améliore l'efficacité des méthodes de data mining en combinant les données possédant des caractéristiques similaires afin qu'une généralisation puisse en être dérivée pour chaque classe ou groupe, ainsi le traitement en batch peut être effectué plutôt qu'un traitement individuel.
- Il fournit une bonne compréhension des similitudes « inhabituelles » qui peuvent se produire une fois que le regroupement est terminé.
- Il fournit une bonne base pour la « recherche des plus proches voisins » (ou *nearing neighbours*) et d'autres des relations plus approfondies.

3.1. CONCEPTS

3.1.1 AMBIGÜITE DES TERMES

L'ouvrage [Carricano et al. 2010] explique qu'en français, « les termes pour décrire le principe de classification d'individus – **typologie, taxinomie, segmentation, classification, catégorisation** – sont relativement ambigus. En anglais, en revanche, le terme *clustering* rend compte à la fois du fait de classer, c'est-à-dire de faire émerger des groupes d'individus, mais également du principe de classification, c'est-à-dire de l'affectation des individus aux différents groupes ».

3.1.2 PARTITIONNEMENT

L'ouvrage [Amini et Gaussier 2013] définit le **partitionnement de documents** comme la répartition en K classes disjointes d'une collection de documents donnée. Il clarifie que les méthodes de partitionnement peuvent être appliqués à toute collection d'objets (texte, images, vidéos, journaux d'opérations de machine, etc.).

Il rappelle également que le terme *clustering* est un anglicisme qui désigne à la fois **partitionnement (hard clustering ou clustering dur)** et une répartition plus souple, en classes non nécessairement disjointes (**soft clustering ou clustering mou**).

L'ouvrage [Amini et Gaussier 2013] présente les **méthodes de partitionnement à plat ou méthodes de réallocation**, fonctionnant suivant le schéma suivant : « (1) identifier des représentants de classes puis (2) affecter chaque document à la classe du représentant (ou centre de classe) le plus proche, ces étapes étant éventuellement répétées. Les méthodes de réallocation reposent bien sur une mesure de similarité (ou de distance) ».

3.1.3 CLASSIFICATION NON SUPERVISEE

Afin d'enlever l'ambiguïté par rapport au terme classification, on parle de **classification non supervisé** pour le clustering et de classification supervisé pour la catégorisation présenté au chapitre 2.2.1.

L'ouvrage [Dreyfus et al. 2008] explique que dans l'apprentissage non supervisé les modèles sont utilisés dans un **but descriptif**. « Les données observées sont constituées d'observations dont on cherche à comprendre la structure : **il n'y a pas de but précis à atteindre**, ni de réponse souhaité ».

3.1.4 CLASSIFICATION NON HIERARCHIQUE

L'ouvrage [Carricano et al. 2010] explique que « les méthodes de classification non hiérarchiques visent à constituer K groupes à partir de n individus de départ. La particularité de ces méthodes, à la différence des méthodes de classification hiérarchique, est que le choix du nombre de groupes se fait en début de processus ».

Il distingue « deux méthodes principales de classification non hiérarchique, qui sont en réalité deux niveaux d'une même approche :

La méthode des centres mobiles : méthode décomposant un ensemble d'individus en un nombre K de classes choisies a priori par un processus itératif. K centres de classes sont initialement choisis (par hasard ou par l'utilisateur de la méthode) parmi n individus. Puis chaque individu est affecté au centre de classe dont il est le plus proche et K groupes sont ainsi formés. Les centres de ces groupes (points moyens) sont alors recalculés. De nouveau, chaque individu est affecté au centre de classe dont il est le plus proche. L'algorithme s'arrête lorsque les centres de classes sont stabilisés.

La méthode des nuées dynamiques : il s'agit d'une généralisation de la méthode des centres mobiles. Par ailleurs, le barycentre de chaque groupe est recalculé lors de l'affectation de chaque individu et non à l'issue de l'affectation de tous les individus au centre de classe initial. Ainsi la convergence est plus rapide et parfois même possible en une seule itération, ce qui peut être utile sur de gros volumes de données ».

Note : au vu de la date (à savoir 2006) de la référence bibliographique utilisée par [Carricano et al. 2010] pour expliquer la méthode des nuées dynamiques, on peut être certain qu'il ne s'agit pas des volumes de données à l'échelle du Big Data.

3.2. NOTION DE SIMILARITE OU DE DISTANCE

Les mesures de distances, comme leur nom l'indique, représente l'éloignement entre deux observations, une valeur élevée indiquant que les observations sont dissimilaires. Cette distance peut être convertie en mesure de proximité en inversant la relation. La principale mesure utilisée est la **distance euclidienne**, qui consiste à calculer la racine carré de la somme des carrées des différences entre les valeurs de chaque variable.

3.2.1 MESURES DE DISTANCE

En plus de la distance euclidienne, il existe bien évidemment d'autres mesures de distance. On présente uniquement les mesures de distance supportées par Mahout ci-dessous [Owen et al. 2011]:

- Manhattan
- Cosinus
- Tanimoto
- Pondérée

La distance **Manhattan** entre deux points est la somme des différences absolues de leurs coordonnées. Cette mesure de distance tire son nom de la structure en grille des rues de Manhattan. Comme tout new-yorkais le sait, vous ne pouvez pas aller à partir de la 2e avenue et de la 2e rue vers la 6e avenue et la 6e rue en marchant tout droit à travers les bâtiments. La vraie distance parcourue est de quatre blocs vers le « nord » et quatre blocs vers « l'est ».

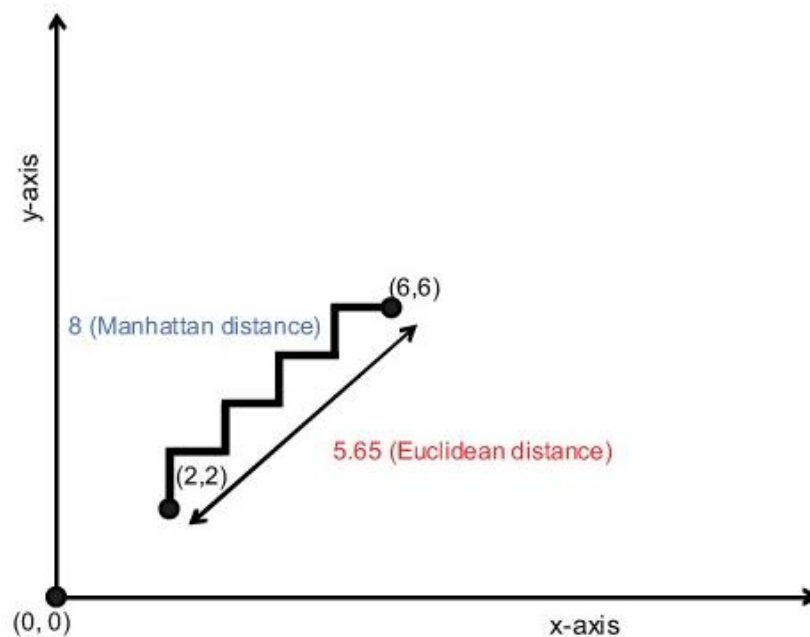


Figure 3 - Mesure de distance Manhattan

La distance **cosinus** mesure l'angle entre deux points considérés alors comme des vecteurs. Lorsque cet angle est petit, les vecteurs sont à peu près orientés dans la même direction, et donc dans un certain sens, les points sont proches. Le cosinus de cet angle est proche de 1 lorsque l'angle est faible et diminue lorsqu'il devient plus grand.

La distance cosinus ne tient pas compte des longueurs des vecteurs. Cela peut bien fonctionner pour certains ensembles de données, mais il peut conduire à un mauvais partitionnement dans d'autres ensembles où les longueurs relatives des vecteurs contiennent des informations précieuses. La distance **Tanimoto**, également connu comme la distance de **Jaccard**, capture l'information sur l'angle et la distance relative entre les points.

La distance **pondérée** est une fonctionnalité avancée dans Mahout qui permet d'affecter des poids à différentes dimensions afin d'augmenter ou de diminuer l'effet d'une dimension sur la valeur de la mesure de distance.

NOTE

Pour information, l'ouvrage [Carricano et al. 2010] liste d'autres mesures de distance disponible sur l'outil SPSS d'IBM et applicable à la classification : par exemple, les distances de Tchebycheff et de Minkowski, et « dans le cadre d'une classification avec de données binaires, on privilégiera les indices de Sokal et Michener, de Rogers et Tanimoto, et celui de Sokal et Sneath ».

3.3. EXEMPLES D'APPLICATIONS

L'ouvrage [Carricano et al. 2010] explique que « **l'analyse topologique** est au cœur de la démarche **marketing**. En effet, on cherche à « identifier des groupes de consommateurs similaires entre eux mais différents d'autres segments. Pour regrouper des consommateurs, on considérera des **variables de segmentation** : géographiques (région, type d'habitat, etc.), sociodémographiques (âge, sexe, taille du foyer, revenus, etc.), psychographiques (style de vie, rapport au temps, personnalité, valeurs, etc.) et comportementales (attitudes, préférences, comportement d'achat, etc.) ».

Egalement utilisé dans le marketing et **l'analyse des réseaux sociaux**, « le **text mining** (extension aux données textuelles du data mining classique) ou plus précisément la **statistique textuelle** a pour objet de découper un texte en unités textuelles (des mots par exemple) puis de regrouper les unités qui sont proches de façon à obtenir des classes homogènes de discours, suffisamment distinctes les unes des autres. Il est ainsi possible, par exemple, d'extraire de grandes classes de mots utilisées par les consommateurs pour parler d'une marque ; un premier groupe de mots fait référence à l'image, un deuxième au rapport qualité/prix, un troisième aux concurrents, etc ».

Autre application la **compression (avec pertes) d'images** ou *lossy image compression* [Mackay 2003]. L'objectif étant d'envoyer la reproduction d'une image avec un minimum d'information (bits). Pour ce faire est on découpe l'image en N *patches* et on affecte chaque *patch* à un *template* de d'image, parmi K templates, se

trouvant dans un alphabet. On envoie ensuite une image proche de l'original avec une liste de N *templates*. La tâche consistant à créer une bonne librairie de *templates* d'image est équivalente à celle de trouver les représentants ou centres de classes.

4. L'ALGORITHME K-MEANS

4.1. INTRODUCTION

Au vu des concepts expliqués au chapitre 3, on peut d'ores et déjà dire que lorsque l'on parle de l'**algorithme k** on parle bien

- d'une **classification non supervisée**
- d'un algorithme de **hard clustering**
- d'une **méthode de partitionnement à plat** ou **de réallocation**
- d'une **classification non hiérarchique** basée sur la **méthode des centres mobiles**
- d'un algorithme reposant sur une **mesure de similarité** ou **de distance**.

4.1.1 PSEUDO-CODE

Ci-après le pseudo-code de la **méthode des centres mobiles** décrite au chapitre 3.1.4 :

Pour commencer, sélectionner (souvent au hasard) et créer K centres de classes.

Tant que tout point change d'affectation à une classe

Pour chaque point dans notre ensemble de données:

Pour chaque centre de classe

calculer la distance entre le centre de gravité et le point

affecter le point au centre de classe le plus proche (distance la plus faible)

Pour chaque groupe ou nouvelle classe, calculer la moyenne des points dans cette classe

affecter le centre de gravité de la moyenne

4.2. LIMITATIONS

Ci-dessous trois limitations mentionnées dans l'ouvrage [Amini et Gaussier 2013], ce dernier en explique d'autres mais elles méritent un effort de compréhension très théorique qui sort du cadre de ce document.

4.2.1 CHOIX DES REPRESENTANTS INITIAUX

« Les deux façons les plus populaires étant

- le tirage aléatoire sans remise de K points de l'ensemble de données
- l'utilisation d'une méthode de partitionnement simple, comme la méthode à une passe (explique également dans l'ouvrage)

La solution finale dépend des représentants (centres de classes) initiaux choisis, et qu'un choix aléatoire de ces représentants rend l'algorithme **non déterministe**.

Pour remédier à ce problème, on exécute en pratique plusieurs fois l'algorithme des k-means et on choisit la partition qui minimise la distance euclidienne ».

4.2.2 CONDITION D'ARRET ET CONVERGENCE

« L'algorithme k-means converge puisqu'à chaque itération la distance interclasse diminue. Il n'y a toutefois aucune garantie que la partition obtenue corresponde à un **minimum globale** (donc à la meilleure solution) de la distance interclasse. En général seul un **minimum local** de cette distance est atteint.

La figure ci-dessous [Harrington 2012] montre le résultat d'une convergence de l'algorithme k-means avec K=3 classes. Comme on peut le constater le partitionnement « n'est pas très bon », ceci est dû choix aléatoire des représentants initiaux et surtout au fait que l'algorithme converge vers un minimum local.

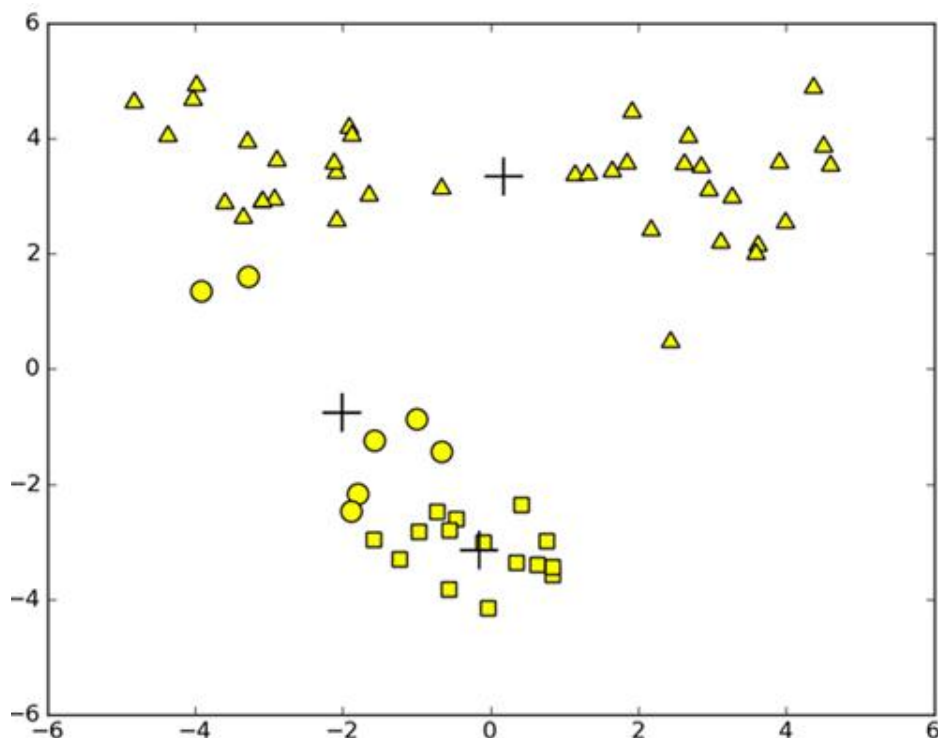


Figure 4 - Exemple d'un mauvais partitionnement ou clustering

Par ailleurs, « la condition d'arrêt porte à la fois sur le nombre d'itérations et la stabilité des classes obtenues. Il est parfois difficile à cause des problèmes d'arrondi informatique, d'imposer que la partition soit stable. La condition sur le nombre maximal d'itérations à réaliser permet de s'affranchir de ce problème d'arrondi. En pratique, quelques dizaines d'itérations suffisent souvent ».

4.2.3 COMPLEXITE

« A chaque itération, il est nécessaire de comparer les N points aux K représentants des classes, ce qui est réalisé en $O(NK)$; la mise à jour des représentants des classes est dominée par $O(N)$, ce qui conduit finalement à une **complexité totale de l'ordre de $O(NKT)$** », T étant le nombre d'itérations maximal admis.

4.3. SOLUTIONS

4.3.1 CANOPY CLUSTERING

« Dans la vraie vie » on ne connaît pas a priori le « meilleur » nombre K de classes. Une catégorie de techniques connues comme **algorithmes de classification approximative** peut estimer le nombre de classes ainsi que l'emplacement approximatif des centres de classes à partir d'un ensemble de données [Owen et al. 2011]. Un de ces algorithmes s'appelle **canopy generation**.

Trouver automatiquement le nombre de classes juste en regardant les données peut paraître passionnant, mais l'algorithme *canopy* doit connaître par avance la taille des classes que l'on cherche, et il trouvera le nombre de classes qui ont à peu près cette taille.

L'algorithme est illustré dans la figure 5. Il commence avec un point et le marque comme faisant partie d'un *canopy* (en haut à gauche), tous les points se trouvant au delà d'un seuil de distance prédéfini T_2 sont supprimés de l'ensemble de données et interdits de former des nouveaux *canopies* (en haut à droite). Les points se trouvant au delà de T_2 mais à une distance inférieure à un deuxième seuil de distance prédéfini T_1 peuvent appartenir à d'autres *canopies* (en bas à gauche). Ensuite les centres de gravité (en bas à droite) sont déterminés et l'algorithme peut fusionner des *canopies* proches.

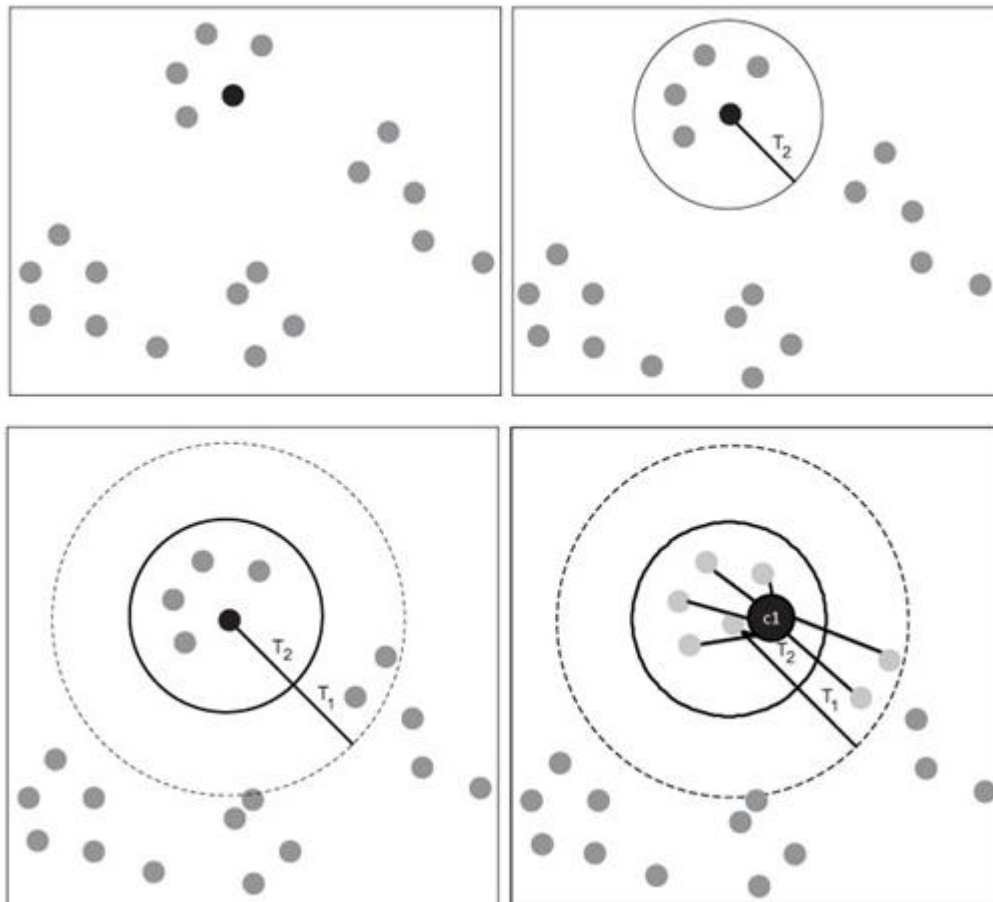


Figure 5 - Algorithme Canopy Clustering

4.3.2 BISECTING K-MEANS

En termes de **qualité de partitionnement** au vu du mauvais résultat de la figure 4, [Harrington 2012] explique que l'une des métriques pour mesurer cette qualité est le SSE (*sum of squared error*) ou somme des carrés des erreurs. Plus le SSE est petit plus les points sont proches de leur centre de classe (ou représentants).

Un moyen de diminuer le SSE serait d'augmenter le nombre K de classes, cependant ce n'est pas forcément l'objectif d'un clustering. Par conséquent, on cherche à améliorer la qualité du partitionnement tout en gardant fixe le nombre K de classes.

Une solution serait de prendre la classe ayant le SSE le plus important et le décomposer en deux classes. Ceci peut être fait en appliquant k-means uniquement aux points appartenant à cette classe avec un K=2. Par rapport à la figure 4, et afin de retrouver le même nombre K de classes du début (ici K=3), on fusionnerait les centres de classes d'en bas.

Cette explication est basée sur la figure 4 c'est-à-dire une visualisation à 2 dimensions seulement, comment fait-on alors avec un cas à 40 dimensions ? L'ouvrage

[Harrington 2012] propose l'algorithme **bisecting k-means** dont le pseudo-code est présenté ci-dessous :

```
Commencer avec tous les points dans une seule classe  
Tant que le nombre de classe K n'est pas atteint  
  Pour chaque classe  
    Mesurer l'erreur totale  
    Appliquer k-means avec K=2 sur une classe  
    Mesurer l'erreur totale après décomposition en deux classes  
  Sélectionner le partitionnement avec la valeur d'erreur totale la moins importante
```

4.3.3 MAPREDUCE

En termes de **complexité**, on constate que le calcul de distance d'un point (vecteur) et un centre de classe est indépendant de celui effectué entre un autre vecteur et un centre de classe. Par conséquent, cela rend l'algorithme **parallélisable**. C'est là que le paradigme MapReduce entre en jeu, il sera expliqué au chapitre 6.

Note : l'algorithme *canopy clustering* présenté au chapitre 4.3.1 est supporté par Mahout et est également parallélisable.

5. K-MEANS AVEC APACHE MAHOUT

Afin de mieux comprendre l'implémentation de l'algorithme k-means dans Mahout, on commencera par expliquer les paramètres de la commande `kmeans`, ensuite on prendra un peu de recul afin de présenter les autres commandes à exécuter avant et après `kmeans` dans une méthode d'exécution plus globale.

5.1. LA COMMANDE KMEANS

Ci-dessous les paramètres de la commande `kmeans`

```
bin/mahout kmeans \  
-i <input vectors directory> \  
-c <input clusters directory> \  
-o <output working directory> \  
-k <optional number of initial clusters to sample from input vectors> \  
-dm <DistanceMeasure> \  
-x <maximum number of iterations> \
```

Les trois premiers paramètres correspondent à la définition des répertoires utilisés par la commande tandis que les trois derniers correspondent bien aux entrées (inputs) de l'algorithme déjà présentés dans les chapitres précédents :

- Le paramètre `-k` est donc le nombre de classes
- Le paramètre `-dm` est la mesure de distance à utiliser
- Le paramètre `-x` est le nombre maximal d'itération (condition d'arrêt)

On constate que le paramètre `-k` est optionnel et cela peut paraître contradictoire avec le fait que l'algorithme a besoin du nombre de classes pour fonctionner. En réalité, le nombre de classes K peut être fourni par un autre algorithme s'exécutant avant k-means, tel que l'algorithme *canopy clustering* expliqué au chapitre 4.3.1.

On rappelle que les mesures de distance supportées par Mahout ont été présentées au chapitre 3.2 : euclidienne, Manhattan, cosinus, Tanimoto et pondérée.

5.2. PROCEDURE DE BOUT EN BOUT

Le schéma ci-dessous présente une méthode qui résume l'exécution des différentes commandes qui seront présentées dans ce chapitre.

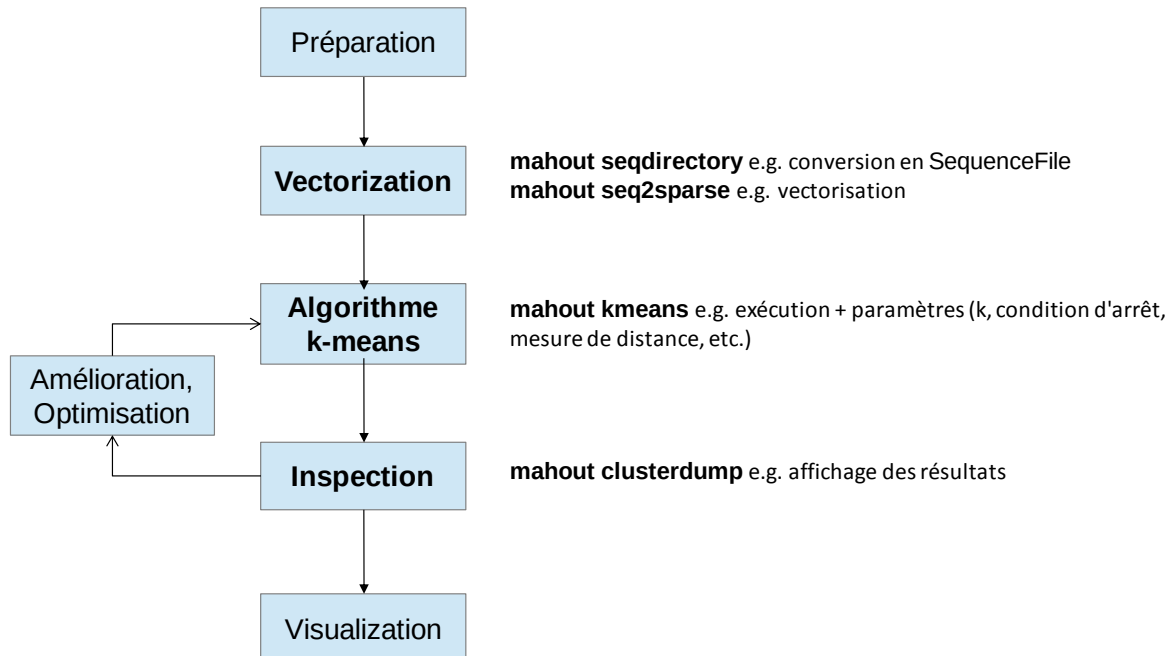


Figure 6 - Procédure de bout en bout

Un exemple de partitionnement de documents basé sur les **articles de l'agence de presse Reuters** sera utilisé afin d'illustrer chaque étape.

Note : les étapes « Visualisation » et « Amélioration / Optimisation » ne seront pas couvertes dans ce document.

5.2.1 PREPARATION

Les articles de l'agence de presse Reuters sont téléchargés à partir du lien suivant :

<http://kdd.ics.uci.edu/databases/reuters21578/reuters21578.tar.gz>

C'est un fichier compressé contenant 22 fichiers en format SGML (extension .sgm). Dans chaque fichier SGML il y a 1000 articles, excepté le dernier fichier qui contient 758 articles.

Par conséquent, l'objectif sera d'effectuer un **partitionnement de 1758 articles ou documents** avec l'algorithme k-means.

Ci-dessous l'exemple d'un seul article en format SGML :

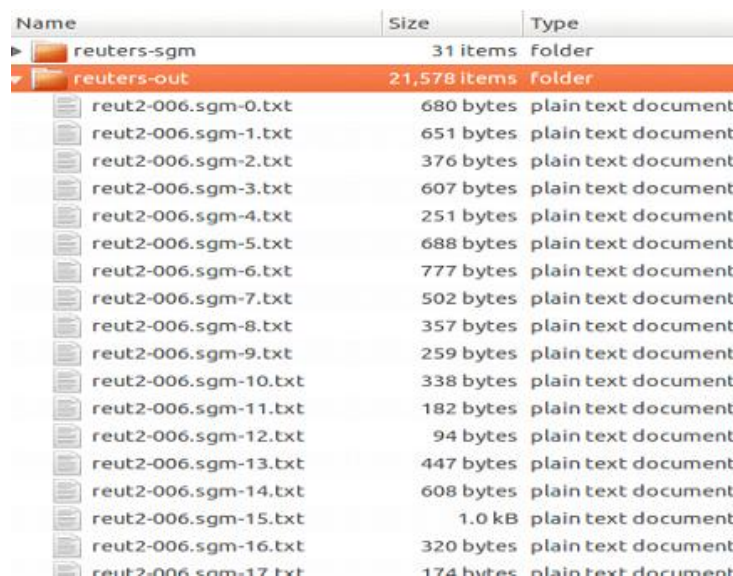
```
<!DOCTYPE lewis SYSTEM "lewis.dtd">
<REUTERS TOPICS="YES" LEWISSPLIT="TRAIN" CGISPLIT="TRAINING-SET" OLDID="5544" NEWID="1">
<DATE>26-FEB-1987 15:01:01.79</DATE>
<TOPICS><D>cocoa</D></TOPICS>
<PLACES><D>el-salvador</D><D>usa</D><D>uruguay</D></PLACES>
<PEOPLE></PEOPLE>
<ORGS></ORGS>
<EXCHANGES></EXCHANGES>
<COMPANIES></COMPANIES>
<UNKNOWN>
&#5;&#5;&#5;C T
&#22;&#22;&#1;f0704&#31;reute
u f BC-BAHIA-COCOA-REVIEW 02-26 0105</UNKNOWN>
<TEXT>&#2;
<TITLE>BAHIA COCOA REVIEW</TITLE>
<DATELINE> SALVADOR, Feb 26 - </DATELINE><BODY>Showers continued throughout the week in
the Bahia cocoa zone, alleviating the drought since early January and improving prospects for the coming temporaao,
although normal humidity levels have not been restored, Comissaria Smith said in its weekly review.
... ..
Final figures for the period to February 28 are expected to be published by the Brazilian Cocoa Trade Commission
after carnival which ends midday on February 27.
Reuter
&#3;</BODY></TEXT>
</REUTERS>
```

Heureusement Mahout intègre une fonction d'extraction d'articles Reuters appelée **ExtractReuters** (fichier JAR de Lucene), il permet d'extraire le contenu marqué en **bleu** ci-dessus et de créer un fichier texte (extension .txt) pour chaque article.

Voici la commande

```
$MAHOUT org.apache.lucene.benchmark.utils.ExtractReuters \
    ${WORK_DIR}/reuters-smg \
    ${WORK_DIR}/reuters-out
```

On obtient alors **1758 fichiers texte** (.txt) correspondant aux extraits des 1758 articles Reuters. Une partie du contenu du répertoire contenant ces fichiers est présentée dans la figure 7.



Name	Size	Type
reuters-smg	31 items	Folder
reuters-out	21,578 items	Folder
reut2-006.sgm-0.txt	680 bytes	plain text document
reut2-006.sgm-1.txt	651 bytes	plain text document
reut2-006.sgm-2.txt	376 bytes	plain text document
reut2-006.sgm-3.txt	607 bytes	plain text document
reut2-006.sgm-4.txt	251 bytes	plain text document
reut2-006.sgm-5.txt	688 bytes	plain text document
reut2-006.sgm-6.txt	777 bytes	plain text document
reut2-006.sgm-7.txt	502 bytes	plain text document
reut2-006.sgm-8.txt	357 bytes	plain text document
reut2-006.sgm-9.txt	259 bytes	plain text document
reut2-006.sgm-10.txt	338 bytes	plain text document
reut2-006.sgm-11.txt	182 bytes	plain text document
reut2-006.sgm-12.txt	94 bytes	plain text document
reut2-006.sgm-13.txt	447 bytes	plain text document
reut2-006.sgm-14.txt	608 bytes	plain text document
reut2-006.sgm-15.txt	1.0 kB	plain text document
reut2-006.sgm-16.txt	320 bytes	plain text document
reut2-006.sgm-17.txt	174 bytes	plain text document

Figure 7 - Fichiers SGML (articles Reuters)

5.2.2 VECTORISATION

Afin d'appliquer le principe de similarité ou de distance expliqué au chapitre 2.3, nous devons transformer chaque document en **vecteur**. Ce vecteur-document se trouvera alors défini dans un espace à n dimensions où chaque dimension est représenté par un **terme**, on l'appelle également l'espace des termes.

La vectorisation des documents texte se fait en deux étapes à travers les commandes correspondantes :

- **seqdirectory**
- **seq2sparse**

CONVERSION DES FICHIERS TEXTE EN SEQUENCE FILE

Nous allons examiner la class Java **SequenceFileFromDirectory** via la commande **seqdirectory**.

Cette classe Java permet de générer une **représentation intermédiaire** du document dans un format **SequenceFile** (défini par Hadoop). Elle exécute cette transformation

à partir du format texte (.txt) du document et place le résultat, voir fichier **chunk-0** dans la figure ci-dessous, dans un répertoire spécifique.

La représentation intermédiaire en format **SequenceFile** est un fichier d'éléments (**clé, valeur**) dont la clé est l'identifiant du document (**doc ID**) et la valeur est le **contenu texte** du document en question.

```
$MAHOUT seqdirectory \  
-i ${WORK_DIR}/reuters-out \  
-o ${WORK_DIR}/reuters-out-seqdir \  
-c UTF-8 \  
-chunk 64 \  
-xm sequential
```

Name	Size	Type
▶ reuters-sgm	31 items	folder
▶ reuters-out	21,578 items	folder
▼ reuters-out-seqdir	2 items	folder
chunk-0	18.4 MB	unknown
.chunk-0.crc	143.8 kB	unknown

Figure 8 - **seqdirectory**: fichier résultat en format SequenceFile

VECTORISATION

Nous allons examiner la class Java **SparseVectorsFromSequenceFile** via la commande **seq2sparse**.

Comme son nom l'indique, cette classe Java, génère des **vecteurs creux** (sparse vector) à partir d'un fichier en format SequenceFile.

Mahout implémente trois classes (Java) de vecteurs: **DenseVector**, **RandomAccessSparseVector** et finalement **SequentialAccessSparseVector**. Cette dernière est optimisée pour une lecture séquentielle et adaptée aux algorithmes tels que k-means [Owen et al. 2011]. Voir le paramètre **-seq** de la commande **seq2sparse** ci-dessous.

```
$MAHOUT seq2sparse \  
-i ${WORK_DIR}/reuters-out-seqdir/ \  
-o ${WORK_DIR}/reuters-out-seqdir-sparse-kmeans \  
--maxDFPercent 85 \  
--namedVector
```

La vectorisation effectuée est basé sur un **dictionnaire de termes** ou **vocabulaire**, voir fichier **dictionary-file-0** dans la figure ci-dessous.

Voici quelques paramètres de la commande **seq2sparse**, la plupart couvrent les **prétraitements linguistiques** appris durant la partie « recherche d'information » du cours NFE204. Ces techniques ne seront pas développés dans ce document, nous invitons le lecteur de consulter l'ouvrage [Amini et Gaussier 2013].

<code>-a</code>	tokenize words e.g. white space with org.apache.lucene.analysis.WhitespaceAnalyzer
<code>-chunk</code>	chunk size in MB
<code>-wt</code>	TF-IDF (default)
<code>-s</code>	Min support value
<code>-md</code>	Min doc freq value
<code>-x</code>	Max doc freq value
<code>-ng</code>	n-gram size
<code>-ml</code>	Min log-likelihood ratio (LLR)
<code>-seq</code>	SequentialAccessSparseVector flag
<code>-norm</code>	Normalisation

Le paramètre `-a` définit la **segmentation** (*tokenisation*) consistant à séparer une suite de caractères en éléments sémantiques, ou mots. On constate que Mahout peut utiliser la librairie Lucene WhitespaceAnalyser qui applique « l'espace » pour la segmentation.

Le paramètre `-wt` utilise par défaut la **pondération TF-IDF** (*term frequency-inverse document frequency*) des termes, c'est-à-dire qu'un vecteur document aura comme coordonnées sa pondération TF-IDF l'espace des termes.

Le paramètre `norm` utilise permet d'effectuer une **normalisation** des vecteurs-documents afin d'éviter le biais consistant à trouver plus d'occurrence des termes à cause des vecteurs-documents très longs. Voir [Owen et al. 2011], chapitre 8.4 pour plus d'information.

Le paramètre `-ng` définit les **n-gram** à appliquer (uni-gram, bigram, trigram, ...), pour des termes tels que « Coca Cola » (bigram), etc.

Le paramètre `-ml` permet de contrôler l'augmentation de la taille du dictionnaire en définissant un **seuil LLR (log likelihood ratio)**. Lorsque l'on passe d'uni-gram à bigram et puis tri-gram on peut obtenir une croissance linéaire de la taille du dictionnaire au lieu d'une croissance exponentiel sans utilisation de ce paramètre.

Mahout effectue les étapes suivantes pour la vectorisation :

1. Les documents texte sont segmentés (tokenized) et placés dans un répertoire intermédiaire spécifique.
2. Le comptage de mots ou *word count* (génération des n-gram) est appliqué de façon itérative sur les documents segmentés. Le dictionnaire des termes est ainsi généré.
3. Pour une pondération TF-IDF, on vectorise les documents segmentés en appliquant un comptage TF (*term frequency*) puis un comptage DF (*document*

frequency) et finalement un calcul de la pondération TF-IDF. Chaque étape crée un répertoire intermédiaire spécifique.

Name	Size	Type
reuters-out	21,578 items	Folder
reuters-out-seqdir	2 items	Folder
reuters-out-seqdir-sparse-kmeans	9 items	Folder
tokenized-documents	4 items	Folder
part-m-00000	15.1 MB	unknown
_SUCCESS	0 bytes	plain text document
.part-m-00000.crc	117.8 kB	unknown
._SUCCESS.crc	8 bytes	unknown
wordcount	4 items	Folder
part-r-00000	993.0 kB	unknown
_SUCCESS	0 bytes	plain text document
.part-r-00000.crc	7.8 kB	unknown
._SUCCESS.crc	8 bytes	unknown
df-count	4 items	Folder
part-r-00000	844.6 kB	unknown
_SUCCESS	0 bytes	plain text document
.part-r-00000.crc	6.6 kB	unknown
._SUCCESS.crc	8 bytes	unknown
tf-vectors	4 items	Folder
part-r-00000	17.0 MB	unknown
_SUCCESS	0 bytes	plain text document
.part-r-00000.crc	133.0 kB	unknown
._SUCCESS.crc	8 bytes	unknown
tfidf-vectors	4 items	Folder
part-r-00000	17.0 MB	unknown
_SUCCESS	0 bytes	plain text document
.part-r-00000.crc	133.0 kB	unknown
._SUCCESS.crc	8 bytes	unknown
dictionary.file-0	824.1 kB	unknown
.dictionary.file-0.crc	6.4 kB	unknown
frequency.file-0	844.6 kB	unknown
.frequency.file-0.crc	6.6 kB	unknown

Figure 9 - Structure du répertoire généré par seq2sparse

5.2.3 EXECUTION DE K-MEANS

On exécute l'algorithme k-means afin d'effectuer un partitionnement de documents en **K=20 classes** (ou **clusters**), en appliquant le cosinus en tant que mesure de distance et un **maximum de 10 itérations**.

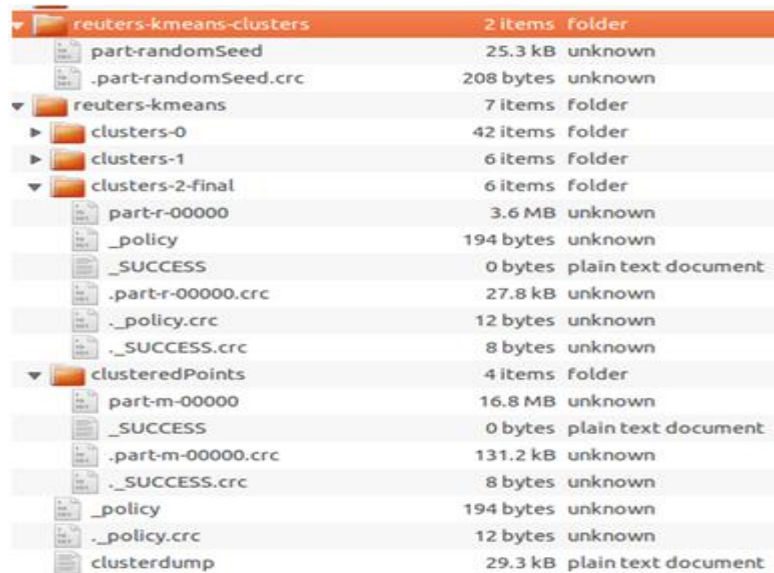
Les centres de classes **initiaux** sont générés avec la classe java `RandomSeedGenerator`. Voir fichier **part-randomSeed** dans la figure 10..

```
$MAHOUT kmeans \  
-i ${WORK_DIR}/reuters-out-seqdir-sparse-kmeans/tfidf-vectors/ \  
-c ${WORK_DIR}/reuters-kmeans-clusters \  
-o ${WORK_DIR}/reuters-kmeans \  
-dm org.apache.mahout.common.distance.CosineDistanceMeasure \  
-x 10 \  
-k 20 \  
-ow \  
--clustering \  

```

Répertoires **clusters-*** sont créés à la fin de chaque itération, ils contiennent des informations sur les classes tels que le centre de classe, l'écart-type, etc.

Le répertoire **clusteredPoints** contient le mapping final clusterID → docID.



reuters-kmeans-clusters	2 items	folder
part-randomSeed	25.3 kB	unknown
.part-randomSeed.crc	208 bytes	unknown
reuters-kmeans	7 items	folder
clusters-0	42 items	folder
clusters-1	6 items	folder
clusters-2-final	6 items	folder
part-r-00000	3.6 MB	unknown
_policy	194 bytes	unknown
_SUCCESS	0 bytes	plain text document
.part-r-00000.crc	27.8 kB	unknown
_policy.crc	12 bytes	unknown
_SUCCESS.crc	8 bytes	unknown
clusteredPoints	4 items	folder
part-m-00000	16.8 MB	unknown
_SUCCESS	0 bytes	plain text document
.part-m-00000.crc	131.2 kB	unknown
_SUCCESS.crc	8 bytes	unknown
_policy	194 bytes	unknown
_policy.crc	12 bytes	unknown
clusterdump	29.3 kB	plain text document

Figure 10 - Structure du répertoire généré par kmeans

5.2.4 INSPECTION

La meilleure façon d'analyser et d'interpréter les résultats de l'algorithme k-means est réaliser une inspection avec la commande **clusterdump**.

```
$MAHOUT clusterdump \  
  
-i ${WORK_DIR}/reuters-kmeans/clusters-*-final \  
  
-o ${WORK_DIR}/reuters-kmeans/clusterdump \  
  
-d ${WORK_DIR}/reuters-out-seqdir-sparse-kmeans/dictionary.file-0 \  
  
-dt sequencefile \  
  
-b 100 \  
  
-n 20  
  
--evaluate \  
  
-dm org.apache.mahout.common.distance.CosineDistanceMeasure \  
  
-sp 0 \  
  
--pointsDir ${WORK_DIR}/reuters-kmeans/clusteredPoints
```

Le paramètre **-o** permet de visualiser le résultat dans un fichier texte appelée **clusterdump**.

Introduction à l'algorithme k-means avec Mahout

En entrée le programme lit le fichier dictionnaire afin de convertir les termes (ou features ID ou dimensions) du vecteur-document en mots.

En sortie il présente un classement par pondération des « top termes » et des informations sur le centre de classes.

A la fin il affiche les valeurs des **distances intra-classe** (que l'on cherche à minimiser) et **inter-classe** (que l'on cherche à maximiser).

```
:VL-7959{n=1601 c=[0.1:0.004, 0.1365:0.012, 0.18:0.006, 0.4:0.004, 0.523:0.012,
0.55:0.006, 0.8:0.004
```

Top Terms:

shares	=>	3.0701310359113743
common	=>	2.055063796147639
stock	=>	1.9897432817212497
said	=>	1.9861087525062155
company	=>	1.5980280315332454
inc	=>	1.5619777712801113
its	=>	1.5518101159667015
dlrs	=>	1.4305196776381142
group	=>	1.394632834482759
mln	=>	1.3522909689813312
pct	=>	1.3290586551973627
stake	=>	1.3190966131983513
share	=>	1.2258259995440852
has	=>	1.1129996320890085
corp	=>	1.090765117482049
buy	=>	1.0501346308167914
outstanding	=>	1.0468410090458982
securities	=>	0.946948390986903
exchange	=>	0.9295935481879802
mar	=>	0.8751847061941134

Weight : [props - optional]: Point:

```
:VL-16919{n=683 c=[0.1:0.010, 0.10:0.022, 0.15:0.012, 0.18:0.013, 0.2:0.010,
0.20:0.013, 0.24:0.014,
```

Top Terms:

pct	=>	3.36537534495922
1	=>	3.004444594264554
bond	=>	2.89077234617264
8	=>	2.573712494153404
manager	=>	2.5478165704718796
lead	=>	2.5447139586397007
coupon	=>	2.5384799090171803
...		
...		
...		

Introduction à l'algorithme k-means avec Mahout

```
:VL-21446{n=1026 c=[0.2:0.006, 0.25:0.007, 0.5:0.031, 00:0.059, 00.05:0.009,
00.07:0.009, 00.12:0.010
```

Top Terms:

said	=>	1.89790412801051
inc	=>	1.4305799771935386
company	=>	1.3819211493691042
its	=>	1.29199122685438
sales	=>	1.2823967536290486
dlrs	=>	1.2325192882768359
Drug	=>	1.1648731208451775
..		
..		

Inter-Cluster Density: 0.640986736268577

Intra-Cluster Density: 0.7052966012163074

CDBw Inter-Cluster Density: 0.0

CDBw Intra-Cluster Density: 18.296697402501465

CDBw Separation: 235.31938537237372

Ci-dessous un extrait de l'ouvrage [Owen et al. 2011] permettant de visualiser les résultats respectifs en appliquant la distance euclidienne et le cosinus. On rappelle que cette analyse n'est pas couverte dans ce document (voir chapitre 1.2).

Listing 10.1. Top 10 terms using k-means clustering with a Euclidean distance measure

Top Terms:		
said	=>	11.60126582278481
bank	=>	5.943037974683544
dollar	=>	4.89873417721519
market	=>	4.405063291139241
us	=>	4.2594936708860756
banks	=>	3.3164556962025316
pct	=>	3.069620253164557
he	=>	2.740506329113924
rates	=>	2.7151898734177213
rate	=>	2.7025316455696204

The top terms in the preceding cluster indicate that it contains documents related to banks. A similar cluster is created using a cosine distance measure in the following listing.

Listing 10.2. Top 10 terms using k-means clustering and a cosine distance measure

Top Terms:		
bank	=>	3.3784878432986294
stg	=>	3.122450990639185
bills	=>	2.440446103514419
money	=>	2.324820905806048
market	=>	2.223828649332401
england	=>	1.6710182027854468
pct	=>	1.5883359918481277
us	=>	1.490838685054553
dealers	=>	1.4752549691633745
billion	=>	1.3586127823991738

They're similar, but the second cluster appears better than the first one. The second cluster identifies *money* as an important term, whereas the first cluster has a common verb *said* as the top term.

Figure 11 - Top termes avec distances euclidienne et cosinus

6. K-MEANS ET MAPREDUCE

Au chapitre 4.3.3 on a constaté que l'algorithme k-means était parallélisable.

Par ailleurs, k-means est un algorithme à 2 étapes:

1. Trouver les points proches d'un centre et les affecter à la classe correspondante.
2. Recalculer les centres de classes avec la moyenne des coordonnées de tous les points dans la classe correspondante.

L'idée étant alors d'affecter la première étape à une fonction **map** et la deuxième étape à une fonction **reduce**.

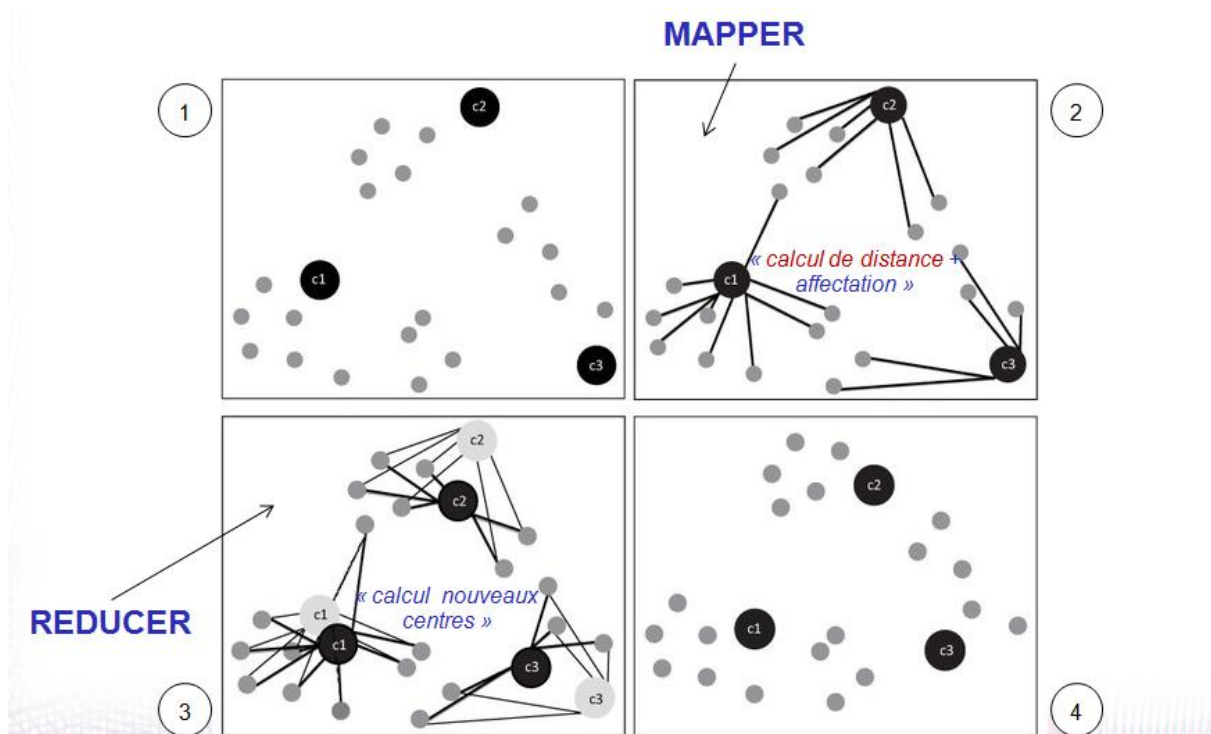


Figure 12 - Une itération Map et Reduce de k-means

6.1. DESIGN MAHOUT

Ci-dessous quelques explications recueillies à partir du site web d'Apache Mahout, sur les classes Java utilisées par Mahout pour l'exécution de l'algorithme k-means en tant que tâche MapReduce.

La classe **KMeansDriver** itère sur les points et les clusters jusqu'à convergence des clusters (cluster ID codé « v_n ») ou jusqu'à ce que le nombre max d'itérations soit atteint.

La classe **KMeansMapper** affecte et affecte chaque point à un centre de classe (ou cluster) selon la mesure de distance spécifiée. **Sortie:** (`clusterID`, `ClusterObservation`)

La classe **KMeansCombiner** reçoit tous les paires (k,v) des mappers et effectue les sommes partielles des vecteurs input pour chaque cluster. **Sortie:** (`clusterID`, `ClusterObservation`).

La classe **KMeansReducer** reçoit les paires (k,v) des combiners, les somme afin de calculer le nouveau centre de classe pour le cluster en question. Création du répertoire « cluster-X » à chaque itération. **Sortie:** (`Vn` | `Cn`, `Cluster`)

La bibliographie Mahout listée au chapitre 8 n'explique pas en détail l'implémentation de l'algorithme k-means en tant que tâche MapReduce. Nous tenterons alors de faire un rapprochement avec un autre design présenté dans le chapitre suivant.

6.2. AUTRES DESIGNS

Cette partie est basée sur le livre blanc technique [Zhao et al. 2009]. Son contenu n'étant pas public alors les pseudo-codes des fonctions **map**, **combiner** et **reduce** ne seront pas présentés explicitement.

L'algorithme décrit dans [Zhao et al. 2009] porte le nom de PKMeans et la description des fonctions **map**, **combiner** et **reduce** est assez proche (comme attendu) de celle de Mahout du chapitre précédent.

Le principe de l'algorithme PKMeans le suivant :

FONCTION MAP

L'ensemble de donnée (dataset), sous forme de **paires (clé, valeur)**, est stocké sur HDFS, chaque paire représente un enregistrement. La clé est un *offset* du début du fichier de données et la valeur est un string du contenu de cet enregistrement.

MapReduce effectue alors un *split* du dataset afin de les distribuer aux mappers.

Pour chaque tâche *map*, l'algorithme PKMeans construit une **variable globale** « centres » de type *array* contenant les informations sur les centres de classes.

Chaque *mapper* calcule la distance de chaque échantillon aux centres de classes et en détermine le plus proche.

Les valeurs intermédiaires retournées sont composées de deux parties : l'index du centre de classe le plus proche (**clé'**) et l'information de l'échantillon (**valeur'**).

FONCTION COMBINER

On utilise alors un *combiner* afin de combiner les résultats intermédiaires provenant d'une même tâche *map*.

Etant donné que ces résultats intermédiaires sont stockés localement il n'y aura pas de communication de ces résultats via le réseau.

La fonction *combiner*

- effectue une somme partielle des valeurs des points affectés à un même centre de classe.
- enregistre également le nombre d'échantillons utilisé pur effectuer les sommes partielles.
- mettra à disposition de la fonction *reduce* (via le réseau) l'index de la classe (**clé**), un string comprenant la somme des échantillons (**valeur**) et le nombre d'échantillons.

FONCTION REDUCE

La fonction *reduce* obtient les résultats du *combiner* de chaque machine (ou *host*).

La fonction *reduce* effectue, pour chaque classe, la somme totale des sommes partielles et le calcul du nombre total d'échantillons appartenant à ladite classe.

Avec ces informations la fonction *reduce* peut en effet calculer les nouveaux centres de classes.

La fonction *reduce* livre l'index de la classe (**clé**) et un string avec les coordonnées du nouveau centre de classe (**valeur**)

6.2.1 RESULTATS EXPERIMENTAUX

Le livre blanc [Zhao et al. 2009] publie des résultats expérimentaux en termes de **speedup** (temps d'exécution en faisant varier le nombre de machines de 1 à 4 et la taille des fichiers de 1Go à 8 Go), **scaleup** (comparaison avec le temps d'exécution de référence correspondant à « 1Go sur 1 machine » des cas suivants : « 2Go sur 2 machines », « 3Go sur 3 machines » et « 4Go sur 4 machines ») et **sizeup** (on fixe le nombre de machines à 1, 2, 3 ou 4 et pour chaque cas on augmente d'un facteur m la taille du fichier par rapport à la taille original).

Configuration des machines : dual-core 2.8GHz avec 4 Go de RAM. Hadoop version 0.17.0 et Java 1.5.0_14.

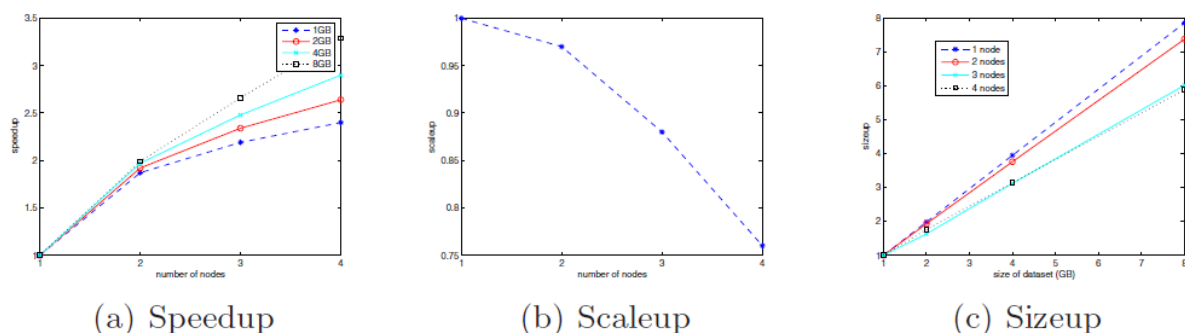


Figure 13 - Résultats expérimentaux [Zhao et al. 2009]

On explique qu'il est difficile d'obtenir un *speedup* « parfaitement » linéaire dû à l'accroissement du trafic réseau avec le nombre de machines dans le cluster. En tout cas, les performances sont bonnes : plus le fichier est gros, meilleur est le *speedup*.

Le *scaleup* montre un très bon passage à l'échelle, c'est-à-dire la capacité d'un système *m fois* plus gros d'exécuter des tâches *m fois* plus importantes dans le même temps de référence que le système original.

Finalement l'algorithme PKMeans montre des très bonnes performances en termes de *sizeup*.

7. CONCLUSION

Au cours du chapitre 1 nous avons expliqué les différences entre l'apprentissage automatique supervisé et non supervisé. L'algorithme k-means appartient à la catégorie « non supervisé » et également à la famille d'algorithmes de *clustering* ou de partitionnement. Après une présentation rapide du projet Apache Mahout, nous avons listé les autres algorithmes de clustering ainsi que les algorithmes de classification (supervisée) et de filtrage collaboratif supportés par Mahout.

Le chapitre 2 nous a permis de revoir les avantages, concepts et caractéristiques générales des algorithmes de clustering. Après clarification entre termes en français utilisé dans le domaine du clustering, les relations entre les notions telles que le partitionnement dur, le partitionnement à plat, la classification non hiérarchique et la méthode des centres mobiles ont été expliquées. Mais plus important encore, la notion de similarité ou de distance a été développée. A titre d'exemple, seules les mesures de distance supportées par Mahout ont été expliquées.

Une présentation théorique de l'algorithme k-means, sans formalisme mathématique, a été donnée au chapitre 4. Son pseudo-code permet de comprendre les étapes et les itérations de l'algorithme. On s'est rapidement attaqué aux limitations bien connues de l'algorithme k-means : le choix aléatoire des centres de classes, la convergence vers un minimum local et sa complexité. Des solutions telles que le *canopy clustering*, l'algorithme *bisecting k-means* et le modèle de programmation MapReduce permettent d'adresser chacune de ses limitations respectivement.

La prise en main de Mahout commence au chapitre 5. Les paramètres de la commande `kmeans` permet de faire les *corrélations* nécessaires avec les notions apprises aux chapitres précédents. Une procédure de bout en bout a été présentée avec un exemple de vectorisation et partitionnement d'articles de l'agence de presse Reuters. Elle nous a permis de connaître d'autres commandes telles que `seqdirectory`, `seq2sparse` et `clusterdump`. La première commande génère des fichiers de paires (clé, valeur) en format SequenceFile, la deuxième permet de paramétrer les prétraitements linguistiques et le troisième de faire une inspection des classes (donc des résultats) générées par l'algorithme k-means.

La procédure expliquée au chapitre 5 ne couvre pas les aspects concernant la distribution de traitements en parallèle sur un cluster de serveurs. Au chapitre 6 on tente d'expliquer deux designs de l'algorithme k-means avec le modèle de programmation MapReduce. D'abord nous présentons les classes Java de Mahout responsables de cette implémentation. Ensuite nous profitons de la publication du

livre blanc technique [Zhao et al. 2009] et son algorithme PKMeans pour décrire les fonctions *map*, *combiner* et *reduce*. On constatera que les principes des deux designs sont pratiquement les mêmes. Finalement, on présente rapidement les résultats expérimentaux en termes de *speedup*, *scaleup* et *sizeup* de PKMeans.

Ce rapport permet d'effectuer une prise en main rapide de Mahout via l'exécution de l'algorithme k-means tout en expliquant au lecteur les concepts, caractéristiques et limitations de l'algorithme. Ces explications sont basées sur une recherche à partir des sources bibliographiques listées au chapitre 9.

8. ANNEX A – CONFIGURATION

Machine : **PC Intel i3 2.93Hz, 4Go RAM**

Système d'exploitation : **Ubuntu 12.04 LTS (64-bit)**

Version Java : **Java 1.7_0.25**

Solution Hadoop : **Cloudera Hadoop 2.0.0 CDH4.0.5**

Version Apache Maven : **3.1.1**

Version Mahout : **0.8**

IDE : **Eclipse “Kepler”** Service Release 1 avec

- plugin **m2e** – Maven integration for Eclipse
- plugin **Subversion** – SVN et connecteur SVNKit

L'installation de Cloudera Hadoop en configuration **pseudo-distribuée** est expliquée dans [Muppalla 2013]

L'installation de Mahout se fait via le plugin Subversion d'Eclipse (ou de NetBeans) en indiquant le repository <http://svn.apache.org/repos/asf/mahout/trunk>, puis on fait un « Import ... » en tant que projet Maven.

Le lecteur pourra également suivre les différentes procédures expliquées dans [Giacomelli 2013] s'il ne souhaite pas installer Cloudera.

9. BIBLIOGRAPHIE ET SOURCES

9.1. OUVRAGES

[Owen et al. 2011] S. Owen, R. Anil, T. Dunning, E. Friedman, « Mahout In Action », Manning Publications, ISBN 978-1-935182-68-9

[Giacomelli 2013] P. Giacomelli « Apache Mahout Cookbook », Packt Publishing, ISBN 978-1-94951-802-4

[Harrington 2012] P. Harrington « Machine Learning In Action », Manning Publications, ISBN 978-1-617290-18-3

[Amini et Gaussier 2013] M.-R. Amini, E. Gaussier, « Recherche d'Information », Eyrolles, ISBN 978-2-212-13532-9

[Dreyfus et al. 2008] G. Dreyfus, J.-M. Martinez, M. Samuelides, M.B. Gordon, F. badran, S. Thiria « Apprentissage Statistique », Eyrolles, ISBN 978-2-212-12229-9

[Carricano et al. 2010] M. Carricano, F. Poujol, L. Bertrandias « Analyse de données avec SPSS » 2^{ème} édition, Pearson, ISBN 978-2-7440-7490-5

[Mackay 2003] D. Mackay « Information Theory, Inference, and Learning Algorithms », Cambridge University Press, ISBN0-521-64298-1

9.2. LIVRES BLANCS TECHNIQUES

[Zhao et al. 2009] W. Zhao, H. Ma, Q. He « Parallel K-Means Clustering Based On mapReduce », CloudCom 2009, LNCS 5931, p. 674-679, Springer-Verlag Berlin Heidelberg

[Anchalia et al. 2013] P. Anchalia, A. Koundinya, N. Srinath « MapReduce design Of K-Means Clustering Algorithm », R V College of Engineering, Bangalore, India, IEEE

9.3. SOURCES WEB

[Muppalla 2013] A. Muppalla « Cloudera Hadoop Installation [Ubuntu Precise] »
<http://4nil.quora.com/Cloudera-Hadoop-Installation-Ubuntu-Precise>

❧ END OF DOCUMENT ❧